AI AGENT AS A FIRST-CLASS COLLABORATOR IN THE OPTIMIZATION PLATFORM

Bachelor Thesis

by

Aftab Khan

Advisor:
Dr. Andreas Karrenbauer

May 11, 2026

Contents

I. Introduction **1**
1. Context . . . 1
2. Challenges . . . 2
3. Goal . . . 3
4. Solution Outline . . . 4

II. Related Work **7**
1. LLMs and Mathematical Modeling . . . 7
2. AI Code Generation . . . 8
3. Human-in-the-loop Architecture . . . 9

III. Concept **13**
1. Overview . . . 13
2. Optimization Modeling . . . 13
3. Large Language Models (LLMs) . . . 15
4. Web Automation . . . 17
5. Chat-User Interaction . . . 18
6. Human-in-the-Loop . . . 19
7. Backend Services . . . 21

IV. Implementation **23**
1. Implementation Overview . . . 23
2. Frontend Integration . . . 24
3. Agent Orchestration Service . . . 26
4. Browser Automation Execution Layer . . . 30
5. Portability and Adaptation to Other Platforms . . . 32

V. Evaluation **35**
1. Evaluation Goals . . . 35
2. Experimental Setup . . . 35
3. Metrics . . . 36
4. Results by Research Question . . . 37
5. Threats to Validity . . . 38
6. Summary . . . 38

VI. Conclusion **39**
1. Summary 39
2. Limitations and Future Work 40

Bibliography **41**

A. Implementation File Structure **45**
1. High-Level Repository Layout 45
2. File Responsibilities 46
3. Cross-Module Execution Path 46

B. Agent State Graph **49**

C. API Endpoint Reference **51**
1. Node Automation Bridge (`AI-Collaborator/puppeteer-server`) 51
2. LangGraph Runtime API Operations (Frontend Usage) 52

D. System Prompt Specification **53**

Sworn declaration

I declare under oath that I have prepared the paper at hand independently and without the help of others and that I have not used any other sources and resources than the ones stated. Parts that have been taken literally or correspondingly from published or unpublished texts or other sources have been labelled as such. This paper has not been presented to any examination board in the same or similar form before.

I further declare that, where AI-based tools were used during the preparation of this thesis (e.g., for language refinement, brainstorming, or code assistance), their use was critically reviewed and transparently documented where required. The academic content, argumentation, and all final decisions remain my own responsibility.

Saarbrücken, _____ _____

Chapter I.

Introduction

1. Context

1.1. Mixed-Integer Linear Programming

Linear Programming (LP) [26] is a mathematical optimization framework in which a linear objective function is optimized under a set of linear equality and inequality constraints. From a methodological perspective, LP provides a rigorous way to represent decision problems in a formal, machine-solvable structure. Owing to this property, LP has become a foundational technique in operations research, applied mathematics, and computer science, with established use in practical domains such as production planning, logistics, energy management, and economic decision support [22].

In many real-world scenarios, however, decision variables are not purely continuous. Integer Linear Programming (ILP) [18] extends LP by constraining selected variables to integer domains, thereby enabling the representation of discrete alternatives, logical dependencies, and combinatorial decisions. Mixed-Integer Linear Programming (MILP) combines continuous and integer variables in a single formulation and is therefore particularly suitable for complex planning and scheduling settings in which continuous quantities (e.g., production volumes) and discrete decisions (e.g., on/off choices, assignment decisions, routing selections) must be optimized jointly. This expressive power explains why MILP is frequently used in both industrial and research contexts despite its increased modeling and computational complexity.

1.2. Optimization Platform

Despite their practical relevance, MILP models are often difficult to construct, validate, and maintain without substantial mathematical and technical expertise. Users must translate domain knowledge into formal constraints, ensure internal consistency of model components, and iteratively debug infeasible or suboptimal formulations. To reduce this barrier, prior work has developed a collaborative web-based Optimization Platform [33, 6, 32, 30] that supports modeling and solving MILP instances through an interactive user interface.

The platform includes a visual drag-and-drop modeler, support for multiple parallel workspaces, and real-time collaboration between multiple users. Beyond usability, these features establish an iterative modeling workflow in which users can incrementally formulate, inspect, and refine optimization models while preserving shared context across collaborators. Consequently, the platform addresses both accessibility and productivity by lowering entry barriers for non-experts and reducing coordination overhead in team-based modeling scenarios.

1.3. Human-AI Collaboration

Recent advances in artificial intelligence, and in particular the emergence of transformer-based large language models (LLMs) [48], have enabled new forms of human-AI collaboration in technical workflows. Rather than functioning only as passive assistants, modern AI systems can participate in iterative problem solving, contextual interpretation, and tool-mediated task execution. In software and engineering contexts, this shift has already demonstrated that language-based interaction can reduce operational friction by allowing users to formulate complex intentions in natural language while delegating implementation-level actions to AI-supported systems.

Motivated by this development, this thesis investigates how AI-driven collaboration can be integrated into optimization modeling workflows to further improve platform usability, modeling efficiency, and support for users with heterogeneous levels of domain expertise. The central premise is that an AI agent can act not merely as a text-based helper but as an active collaborator embedded in the modeling environment, capable of proposing context-aware model modifications while remaining under explicit human supervision.

2. Challenges

Although the current optimization platform already provides a strong collaborative and user-oriented foundation, several limitations remain that constrain modeling efficiency, accessibility, and long-term usability in practice. In particular, the following challenges motivate the need for an AI-assisted collaboration layer in the proposed system:

(i) **Inefficiency in Complex Modeling Workflows:** The visual drag-and-drop interface improves usability for small to medium formulations, but can become increasingly time-intensive for large or highly structured MILP models. Repetitive modeling actions, frequent structural adjustments, and manual consistency checks introduce significant overhead. As model complexity grows, this interaction pattern can reduce development velocity and increase the risk of modeling errors that are difficult to detect without systematic support mechanisms.

(ii) **Accessibility and Learning-Curve Barriers:** The platform offers a rich set of functionalities, but this feature depth can create a substantial onboarding burden for users with limited optimization background. New users must simultaneously

understand domain concepts, interface mechanics, and modeling conventions before they can contribute effectively. Consequently, the initial learning curve remains steep, which may limit adoption and delay productive use in educational and applied settings.

(iii) **Limited Real-Time Guidance for Individual Users:** While collaborative sessions with multiple human participants can provide peer support, users working independently often lack immediate assistance during model construction, debugging, or refinement. In such scenarios, unresolved uncertainty can interrupt workflow continuity and lead to trial-and-error interactions that are both slow and error-prone. A context-aware assistant capable of delivering on-demand guidance and actionable model suggestions can therefore improve both user confidence and overall modeling quality.

3. Goal

The objective of this thesis is to design, implement, and analyze an AI agent that functions as a first-class collaborator within an existing collaborative optimization platform. In contrast to conventional assistant paradigms that remain external to the modeling process, the proposed agent is embedded directly into the shared workspace and participates through the same collaboration interface used by human users. Operationally, the agent receives natural-language instructions, interprets them with respect to the current model state, and generates candidate modifications to model structures, constraints, and related workspace artifacts.

A central design principle of this work is human-centered control and accountable collaboration. Consequently, agent-generated modifications are treated as provisional artifacts rather than immediately committed updates. Users can inspect proposed changes, approve or reject them, and provide iterative feedback that is incorporated in subsequent generation cycles. This interaction model establishes a human-in-the-loop workflow that combines automation with user supervision and preserves human authority over final model decisions.

To improve reliability, the architecture incorporates a validation stage between generation and commitment. This stage performs syntactic and structural checks on generated model artifacts and filters invalid, inconsistent, or hallucinated outputs before they affect the shared workspace. As a result, the system aims to maintain modeling correctness while still benefiting from rapid AI-assisted iteration.

At a research level, this thesis seeks to answer the following research questions:

- Can an integrated, stateful, and supervised agent architecture reduce modeling effort for routine and repetitive tasks?
- Can such an architecture lower entry barriers for users with limited optimization expertise?
- Can it improve the efficiency and quality of collaborative MILP model development?

The overarching goal is therefore not only to introduce AI-generated assistance, but to operationalize it as a reliable collaboration mechanism within a production-oriented optimization environment.

4. Solution Outline

This section provides a structured overview of the proposed system architecture and its interaction workflow. At a high level, the solution combines four tightly coupled elements: (i) a collaborative modeling workspace, (ii) an LLM-based agent that interprets user intent, (iii) an iterative feedback mechanism for human supervision, and (iv) a validation layer that enforces model-level correctness constraints before commitment. Together, these components form a stateful human-AI collaboration loop designed for optimization modeling tasks.

4.1. AI-Agent Collaborator

The AI agent is integrated into the Optimization Modeler as a workspace participant rather than an external utility. After a user creates or opens a model, the agent can be invited through the existing collaboration infrastructure and becomes part of the shared session context. This design choice preserves architectural consistency with human collaboration while enabling the agent to access the same model state, interaction history, and relevant workspace metadata required for context-aware assistance.

Functionally, the agent acts as an execution-capable collaborator: it receives user instructions, maps them to model-level operations, and generates candidate changes in a form that can be inspected within the same collaborative environment. By embedding the agent directly into the session, the system avoids context fragmentation between separate tools and supports a unified modeling workflow.

4.2. Iterative Feedback

The collaboration workflow follows an iterative proposal-and-review pattern. Users express modeling intentions in natural language, after which the agent generates a candidate update based on the current workspace state. Instead of committing changes immediately, the system exposes the proposal for explicit user evaluation. Users can approve the proposal, reject it, or provide corrective feedback that is incorporated into subsequent iterations.

This interaction model serves two goals simultaneously: it leverages AI assistance to accelerate routine modeling operations, and it preserves human control over semantically critical decisions. Iterative refinement also enables progressive convergence toward higher-quality model formulations, especially in cases where the initial prompt is ambiguous or under-specified.

4.3. Safety and Validation

A dedicated validation stage is executed between agent generation and final model commitment. This stage performs syntactic and structural checks to ensure that generated artifacts are compatible with platform constraints and do not violate core modeling assumptions. Outputs that fail validation are rejected or returned for further refinement, preventing malformed or hallucinated updates from being applied to the shared workspace.

From a systems perspective, this validation layer is essential for reliable AI integration in optimization environments: it acts as a safeguard against invalid transformations while maintaining workflow continuity for valid proposals. Consequently, safety is treated as an operational property of the collaboration pipeline rather than a post hoc correction step.

Chapter II.

Related Work

1. LLMs and Mathematical Modeling

1.1. End-to-End Modeler and Solver Archirecture

Extensive research has been conducted at the intersection of Large Language Models (LLMs) and mathematical optimization. A prominent example is **OptLLM** [54], a framework designed to bridge natural language descriptions and specialized solvers. OptLLM utilizes a modular architecture consisting of three primary components: the *Interaction Refinement Module*, which processes user queries and handles multi-round clarifications; the *Converter Module*, which includes a formulator to translate text into mathematical structures and a coder to generate executable code; and finally, the *Responser Module*, which executes the model via external solvers and interprets the results for the user. This architecture demonstrates the feasibility of integrating disparate modules to transform an LLM into a comprehensive end-to-end optimization system.

1.2. Natural Langauge to Formal Specification

Another significant contribution is **LLMOPT** [20], a learning-based framework that focuses on boosting the generalization of LLMs across various optimization types. LLMOPT introduces a universal *Five-Element Formulation* that reduces complex problem descriptions into five core components: sets, parameters, variables, objectives, and constraints. This structured representation allows the system to generate consistent solver code across diverse domains, including linear, nonlinear, and combinatorial optimization.

1.3. LLM Benchmark for Optimization Modeling

While the aforementioned frameworks focus on generation, **Mamo** [17] provides a specialized benchmark to evaluate these modeling capabilities. Mamo consists of 1,209 meticulously curated questions spanning Ordinary Differential Equations (ODEs), Linear Programming (LP), and Mixed-Integer Linear Programming (MILP). Unlike traditional result-oriented metrics, Mamo utilizes a "model generation and numerical validation" approach. An LLM is tasked with formulating a model from the dataset, which is then

executed by a solver; the resulting numerical answer is compared against the provided ground truth. This benchmark is particularly valuable for the evaluation phase of this thesis, as it provides a standardized methodology for measuring the accuracy of AI-generated mathematical models.

2. AI Code Generation

2.1. Current State

Since the introduction of the Transformer architecture [48], there has been a paradigm shift in the field of automated software engineering. While Transformers were initially celebrated for their success in Natural Language Processing (NLP), their ability to model long-range dependencies and complex structural patterns has led to significant advancements in Large Language Models (LLMs) for code generation.

Extensive research has explored the transition from general-purpose text generation to domain-specific code synthesis. As detailed in the comprehensive survey by Jiang et al. [21], modern frameworks leverage massive datasets of source code to perform tasks such as code completion, bug detection, and natural-language-to-code translation. The survey categorizes state-of-the-art models—including GPT-4, CodeLlama, and StarCoder—and examines the underlying training objectives, such as "Fill-in-the-Middle" (FIM) and instruction tuning, which are essential for understanding the non-linear structure of programming languages.

2.2. Coding Agents

Recent advancements in AI-powered coding assistants—such as GitHub Copilot [37], Google Gemini CLI [15], and Anthropic's Claude Code [2]—demonstrate how generative AI significantly enhances software development workflows. These coding agents operate as collaborative tools that integrate directly into development environments, where they utilize Large Language Models (LLMs) fine-tuned on vast code and documentation corpora [53]. Typically, users write natural language prompts or partial code snippets; the agent responds by generating, refactoring, or completing code, suggesting best practices, and occasionally identifying bugs.

Most modern coding agents rely on transformer-based architectures pre-trained on publicly available open-source repositories, and are further tuned for developer intent and task relevance through techniques such as Reinforcement Learning from Human Feedback (RLHF) and tool integration APIs [53]. For example, GitHub Copilot, powered by OpenAI's Codex model, integrates seamlessly into IDEs and proactively suggests entire code blocks in real-time [37]. Google's Gemini CLI and Anthropic's Claude Code Agent similarly offer advanced contextual understanding, enabling cross-file reasoning and multi-modal support such as code explanation, completion, or test synthesis.

Despite their power, these tools exhibit key limitations when applied to mathematical optimization modeling [14]. Primarily, their training prioritizes general-purpose software

engineering tasks—such as web development, scripting, or library usage—rather than symbolic reasoning or formal specification needed for optimization problems. As a result, these agents may struggle with generating accurate and semantically valid mathematical models, including algebraic constraints, variable declarations, and objective functions, particularly when such constructs must strictly adhere to domain-specific formats. Furthermore, their validation mechanisms are largely confined to code execution or basic unit tests, whereas optimization modeling requires both syntactic correctness and adherence to deeper mathematical requirements.

Thus, while coding agents have greatly improved code generation and developer productivity in mainstream programming, their architecture and underlying datasets are not inherently optimized for the highly structured, domain-specific tasks characteristic of mathematical optimization modeling.

2.3. LLM Benchmarking for Code Generation

As Large Language Models (LLMs) continue to evolve, the demand for more rigorous evaluation methodologies has led to the development of advanced benchmarking frameworks. One such framework is **EvalPlus** [28], which addresses the limitations of traditional code generation benchmarks.

EvalPlus builds upon established datasets, most notably *HumanEval* [8] and *MBPP*. While these original benchmarks provided a foundational method for measuring functional correctness, their test suites are often sparse and fail to catch edge-case failures, allowing incorrect code to pass as correct. EvalPlus mitigates this by automating the synthesis of thousands of additional test cases for each problem. By significantly increasing the test-case density—often by a factor of 80x compared to the original suites—EvalPlus creates a more robust evaluation environment that exposes "false positives" in LLM-generated code.

3. Human-in-the-loop Architecture

3.1. Agentic Development Environments

Recent years have witnessed the emergence of agentic Integrated Development Environments (IDEs) that proactively embed AI agents into the user's workflow. Notable examples include **Cursor** [9], Google's **Antigravity** [3], and the Copilot extensions for Visual Studio Code [49]. These environments aim to facilitate a tight human-in-the-loop collaboration by making intelligent assistants easily available during the development process.

For instance, **Cursor** offers an AI agent that not only generates and edits code but also explains suggestions, processes natural language commands, and supports high-level refactoring through conversational interfaces directly within the code editor. Google's **Antigravity** further pushes this concept by providing contextual tool recommendations and automated fixes based on the developer's current task, often prompting the user

for confirmation or guidance before performing significant actions. Similarly, Copilot in Visual Studio Code integrates an AI agent trained on vast open-source codebases, providing real-time code completions and suggestions that the human developer can freely accept, reject, or modify as needed.

In all these platforms, the human developer remains a principal decision-maker—reviewing, editing, or vetoing AI-generated proposals. This forms a continuous feedback loop where the agent assists, but all critical actions are subject to user oversight and approval, embodying the essence of "human-in-the-loop" design for code authoring environments.

However, while these agentic IDEs excel in integrating AI assistance for general-purpose software development, their capabilities are typically confined to code generation, editing, and debugging. They are not inherently equipped for specialized tasks such as mathematical modeling, formal specification, or symbolic reasoning outside traditional software engineering domains. As a result, their human-in-the-loop mechanisms and agentic workflows remain primarily limited to mainstream coding activities, rather than broader, domain-specific applications like mathematical optimization modeling.

3.2. Domain-Specific Agentic Systems

Recent advancements have witnessed the emergence of domain-specific agentic systems—AI assistants designed with a focus on particular industries, products, or workflows, rather than general-purpose programming. These agents are embedded into platforms to automate, streamline, and augment specialized tasks through close integration, fine-tuned models, and continual user-driven feedback.

One notable example is **v0** [47], an AI-powered design assistant specializing in rapid user interface (UI) prototyping. V0 enables designers to describe layouts and components in natural language, then instantly generates production-grade code tailored to target frameworks like React or Vue. The system's effectiveness is a result of fine-tuning large language models on datasets encompassing real-world design systems, annotated UIs, and established coding conventions, ensuring that outputs are compatible with prevailing design and engineering standards.

Industry-leading productivity suites have also embraced agentic AI. Both the **Microsoft Copilot suite** [29] (integrated into Word, Excel, PowerPoint, Outlook, etc.) and the AI features within the **Google Workspace** [51] suite (Docs, Sheets, Slides, Gmail, and more) exemplify domain-specific assistants for business documentation and office workflows. These agents draft emails, generate meeting summaries, synthesize data into insights, automate spreadsheet analyses, and manage scheduling—all by leveraging AI models refined with proprietary enterprise data, fine-tuned user interactions, and organization-specific context. Their integration is further enhanced by enforcing strict access controls, supporting privacy-preserving computations, and continuously learning from authenticated user behavior within organizational environments.

In the academic and technical writing domain, platforms like **Overleaf** [1] have integrated targeted AI helpers that support LaTeX authors by providing syntax correction, auto-completion of citations, formatting suggestions for mathematical content, and real-time collaborative editing tips. These capabilities are achieved by specializing large

language models with academic writing data and technical document corpora, ensuring that suggestions are both technically accurate and contextually appropriate.

In addition to these deep integrations, a ubiquitous trend is the proliferation of AI helper chatbots directly on product, SaaS, and documentation pages. A representative example is the AI assistant available on the **Assistant-UI** documentation page [19], where an integrated chatbot guides users through setup processes, answers technical questions, and helps to resolve onboarding or configuration issues in real time. Such conversational agents are designed to offer immediate, context-aware assistance—addressing product queries, helping new users get started, and providing proactive troubleshooting or feature recommendations. Their models are typically trained on product documentation, frequently asked questions, and anonymized user interactions, ensuring that responses are both relevant and tailored to the specific service. Many vendors fine-tune these models to reflect evolving user needs and workflows, continually enhancing the effectiveness and relevance of automated customer support.

Collectively, these domain-specific agentic systems illustrate a shift toward AI agents that are purpose-built, data-driven, and closely aligned with the specialized tasks and environments of their target users. Through model fine-tuning, deep platform integration, and continual feedback, they offer contextually intelligent support that goes well beyond what is achievable with general-purpose language models.

3.3. Magentic-UI

Magentic-UI [31] is a human-in-the-loop (HITL) research prototype by Microsoft Research, designed to facilitate complex web-based tasks through close collaboration between the user and an AI agent. In contrast to fully autonomous agents, Magentic-UI foregrounds the user's role as an active collaborator. The system processes natural language queries from the user, interprets intent, and generates a structured, multi-step plan for task execution.

Central to Magentic-UI are its advanced interaction mechanisms: *co-planning* and *action guards*. Through co-planning, the agent transparently presents its proposed sequence of actions, allowing the user to review, modify, approve, or halt steps before anything is carried out. Action guards further require explicit human approval for any pivotal or potentially sensitive operations, such as sharing confidential data or triggering transactions. This layered approach ensures user oversight at key decision points, fostering transparency and safety throughout the process. By embedding human review and iterative feedback into the agent's workflow, Magentic-UI exemplifies contemporary best practices in human-in-the-loop AI architectures—especially for web-based, agentic environments—highlighting the synergy of AI assistance and maintained human authority.

Chapter III.

Concept

1. Overview

The proposed concept models the AI agent as a *first-class collaborator* in the existing Optimization Platform instead of treating it as an external helper. This design decision follows the same collaboration paradigm already established for human users in prior platform work [33, 6, 32, 30]: the agent joins a shared workspace, observes the current model state, proposes modifications, and interacts with users through the same operational context.

At a system level, the concept consists of five tightly integrated layers:

(i) **Optimization workspace layer**, where users and the agent operate on a shared model state.

(ii) **LLM reasoning layer**, which translates natural-language instructions into domain-specific model edits.

(iii) **Interaction layer**, which provides a persistent chat interface inside the modeler.

(iv) **Execution layer**, which applies generated edits to the web modeler through controlled browser automation.

(v) **Control and validation layer**, which enforces human oversight and structural correctness before final acceptance.

This chapter discusses each conceptual pillar, compares relevant alternatives, and justifies the selected design with respect to the implementation constraints of the platform.

2. Optimization Modeling

The Optimization Platform is centered around a visual modeler for mixed-integer linear programming (MILP), where models are represented as structured markup and rendered as editable workspace content. In this environment, modeling is inherently stateful: every change affects subsequent editing, collaboration, validation, and solving steps.

Multi-user collaboration is realized through a shared workspace session with role-based access links (write/edit versus read-only), following the collaboration model established

in earlier platform iterations [33, 6]. Once users join the same workspace, all participants operate on a common model state instead of maintaining independent local copies. Conceptually, each edit is treated as an event against this shared state and is propagated to connected collaborators through the session channel, so that structural changes in the modeler are reflected across participants with minimal delay.

Change propagation therefore follows a *single-source-of-truth* pattern: updates are applied to the active workspace representation, broadcast to collaborators, and rendered in their current modeler context. This reduces merge conflicts typical of file-based workflows and enables synchronous co-modeling. At the same time, role separation remains important: users with write permissions can produce state transitions, while read-only participants observe the same evolving model without mutating it.

From a conceptual perspective, this has two direct implications for AI integration. First, the agent must operate on the *current workspace state* rather than on isolated prompts. Second, model updates must preserve structural consistency with the platform-specific schema used by the modeler. Consequently, the agent is not designed as a generic text completion system; it is designed as a context-aware transformation component that produces valid, executable modeling artifacts.

The proposed concept therefore adopts an in-place collaboration strategy: the AI agent is invited into the same workspace channel as human collaborators and acts on the same editable model representation. This avoids synchronization issues between separate tools, preserves the platform's existing collaboration semantics, and keeps the user in a familiar workflow.

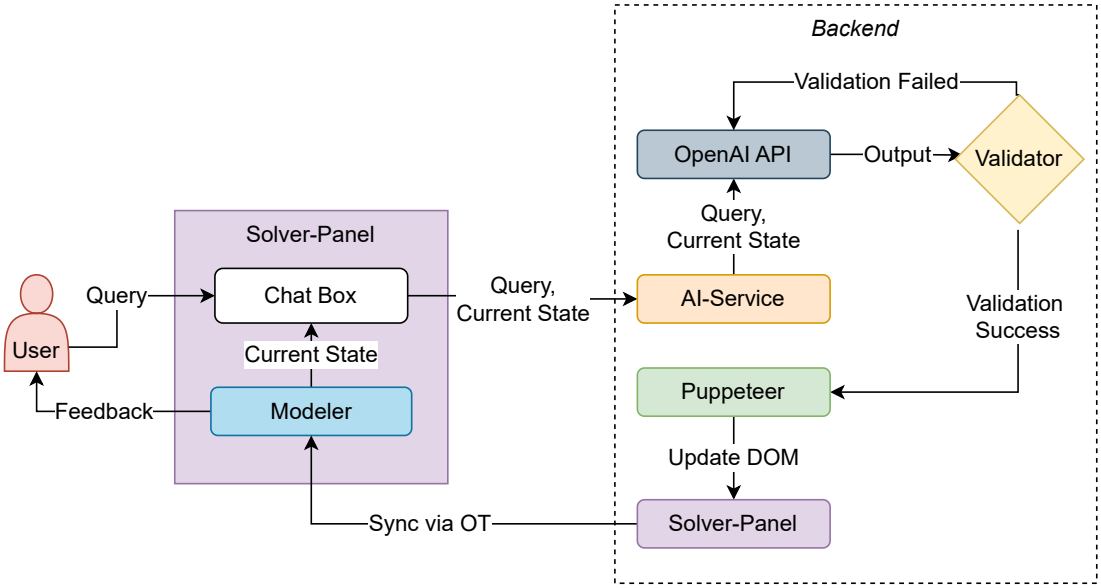

Figure III.1.: Optimization model with AI agent integrated into the collaborative workspace.

3. Large Language Models (LLMs)

Large Language Models (LLMs) [25] provide the semantic interface between human intent and formal model edits. In this thesis, the LLM layer is responsible for mapping natural-language requests to platform-specific markup while respecting optimization semantics and syntactic constraints. Unlike standard code completion tasks, this mapping requires both domain-aware reasoning and strict output-shape control.

At a technical level, contemporary LLMs are predominantly transformer-based sequence models that predict the next token conditioned on prior context [48]. During pretraining, models ingest large-scale text and code corpora and learn statistical regularities of language, structure, and reasoning patterns. After pretraining, they are typically adapted through instruction tuning [55] and preference alignment so that they better follow user intent, produce more useful responses, and reduce unsafe behavior. In practical use, user prompts are converted into token sequences, processed through stacked self-attention and feed-forward layers, and decoded into output tokens that form the final response.

The current LLM landscape is characterized by rapid iteration along three dimensions: (i) *capability*, including stronger reasoning and code generation performance; (ii) *interaction modality*, including support for text, code, and increasingly multimodal inputs; and (iii) *deployment model*, spanning proprietary hosted APIs and open-weight/self-hosted alternatives [21, 53, 56]. For engineering applications, this means model choice is no longer purely about benchmark scores; it is also about latency, cost, controllability, context-window limits, and integration ergonomics.

In this context, an *LLM API* is the operational interface that allows external systems to call a hosted model service programmatically. Instead of running the model weights locally, the client sends structured requests (e.g., messages, system instructions, tool schemas, generation parameters) to a remote endpoint and receives generated outputs, often with optional streaming. A representative example is the OpenAI API [5], where applications send chat-completion style requests and receive token-generated responses that can be consumed in real time. Conceptually, APIs decouple model consumption from model hosting: the application focuses on orchestration logic, while model serving, scaling, and infrastructure maintenance remain with the provider.

For this thesis, that API-based design is particularly beneficial. It enables fast experimentation across models, simplifies integration with orchestration frameworks, and supports tool-calling patterns required for stateful, human-in-the-loop workflows. At the same time, API usage introduces practical constraints (cost, quota, rate limits, external dependency), which must be handled at the system-design level.

3.1. Alternatives

Several alternatives are plausible for this layer:

- **Single-provider LLM integration:** Directly binding the system to one model vendor (e.g., only OpenAI [5] or only Anthropic [46]) simplifies initial setup but increases vendor lock-in and limits model experimentation.

- **Local/self-hosted models only:** This improves deployment control and potentially data governance, but may reduce generation quality for complex optimization prompts unless substantial infrastructure and tuning effort are invested. Solutions such as Ollama [36] and similar local hosting frameworks can enable on-premises LLM inference, increasing privacy and autonomy, but they typically require significant local compute resources as a trade-off, and may lag in access to the latest frontier models.
- **Model-provider abstraction :** A dedicated routing/proxy layer under full application control can unify multiple vendor APIs behind one interface, but introduces an additional service to operate, monitor, and secure.

3.2. Choice

The implemented concept uses **OpenRouter** [38] as the model-provider abstraction layer. OpenRouter is a hosted routing service that exposes an OpenAI-compatible API [4] surface while allowing requests to be dispatched to different underlying model providers through a unified interface. In practice, the client sends a standard chat-completion style request to OpenRouter together with a model identifier; OpenRouter then forwards the request to the selected backend model and returns the generated response in a compatible format.

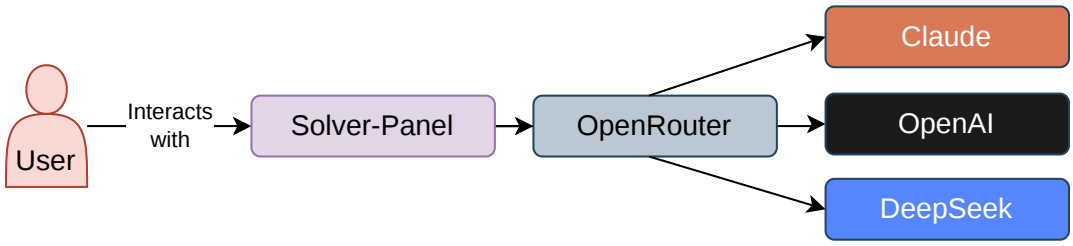

Figure III.2.: OpenRouter as a model-provider abstraction layer routing requests to different LLM backends.

Relative to alternatives such as a custom provider gateway or a LiteLLM [27]-based self-managed abstraction, this choice favors lower operational overhead in the thesis context: multi-model routing is available immediately without introducing an additional in-house proxy service that must be deployed and maintained. In this project, the choice is motivated by the following concrete implementation requirements:

(i) **Provider-level model portability:** The implementation can change model backends by updating the model identifier and provider configuration, while preserving the same request contract in the orchestration layer.

(ii) **Fast experimentation across model variants:** The project can evaluate different cost/latency/quality trade-offs during development without redesigning the surrounding tool-calling workflow.

(iii) **Lower operational burden than self-managed routing:** Compared with running an internal proxy abstraction layer, the hosted routing approach avoids additional service deployment, monitoring, and maintenance tasks in this thesis scope.

In short, the selected design optimizes for adaptability and controllability, which are more important for this thesis than maximizing benchmark performance on generic coding tasks.

4. Web Automation

The AI agent must not only generate edits but also apply and retrieve workspace content in a live web session. This requires a browser automation layer capable of authenticated login, workspace navigation, DOM [10] extraction, and deterministic DOM updates.

Conceptually, web automation is the programmatic control of a browser instance to reproduce user-level interactions with a web application. Instead of invoking internal functions directly, an automation script executes actions at the interface level (e.g., open page, fill form fields, click controls, read DOM state), which makes the behavior closer to real end-user operation. This is particularly relevant when key platform functionality is embedded in client-side logic and collaborative UI workflows rather than being fully exposed through stable backend APIs.

Operationally, a web automation runtime launches a browser process, opens a page context, and then performs a sequence of scripted actions. After each action, the runtime can inspect page state through selectors and JavaScript evaluation, enabling closed-loop execution: observe current state, execute next action, and verify effect. In this thesis, that mechanism is used not only for one-time setup (authentication and workspace joining), but also as a continuous execution environment where the agent can read the current model representation and apply validated updates back to the modeler DOM.

From an architectural perspective, this browser session functions as the agent's *environment interface*. The LLM decides *what* model transformation should be performed, while the automation layer is responsible for *how* that transformation is enacted in the live collaborative workspace. This separation improves modularity: reasoning remains model- and prompt-centric, whereas execution remains UI- and state-centric.

4.1. Alternatives

Common alternatives include:

- **Direct internal API integration:** If all required model operations are exposed through stable backend APIs, browser automation can be avoided. In this project, however, the collaboration and model-editing logic is heavily UI-centric.
- **Selenium [43]-based automation:** Mature and widely supported, but often heavier for modern JavaScript-centric workflows.

- **Playwright [12]-based automation:** Strong multi-browser support and robust testing ergonomics, but migration would require additional adaptation effort for existing script patterns.

4.2. Choice

The implemented concept uses **Puppeteer** [40]. For the current platform architecture, this is advantageous because:

(i) **Native Chromium [16] focus:** The modeler workflow is browser-centric and JavaScript-heavy; Puppeteer provides tight control over page navigation and DOM operations in this context.

(ii) **Low conceptual overhead:** The API supports direct element evaluation and straightforward automation scripts for login, session joining, and model updates.

Given the thesis objective (functional AI collaboration in a real platform), this choice offers the best trade-off between engineering effort and operational capability.

5. Chat-User Interaction

For first-class collaboration, the user-agent channel must be embedded directly in the modeler, maintain conversational context, and support iterative refinement instead of one-shot commands. In practice, chat interaction in this system is not treated as an auxiliary feature; it is the primary coordination mechanism between user intent and agent execution. Users describe desired model changes in natural language, inspect generated responses in the same interface, and continue with clarification or corrective feedback when needed.

From a conceptual standpoint, this interaction model has three requirements. First, the chat must remain *workspace-aware*, so that responses are grounded in the current model state rather than in isolated prompts. Second, it must support *persistent multi-turn state*, because optimization edits frequently require incremental negotiation (e.g., add a variable, then constrain it, then revise objective coefficients). Third, it must support *low-friction UX patterns* (message history, typing and loading states, structured rendering, clear action affordances) so that users can focus on model logic instead of interaction mechanics.

5.1. Alternatives

Three major alternatives were considered:

- **External chat application:** Easy to prototype, but separates communication from the modeling context and introduces coordination friction.
- **Custom chat UI from scratch:** Full control over behavior and styling, but high implementation and maintenance cost.

- **Minimal command input widget:** Reduced complexity, but insufficient for long-form clarification and multi-turn collaboration.

5.2. Choice

The selected implementation embeds an in-modeler conversational UI based on **assistant-ui** with runtime integration for stateful agent communication [19, 24]. The frontend implementation combines **React** [42], **shadcn/ui** [44]-style component primitives, and **Tailwind CSS** [45] utility styling. Together, these layers provide a composable foundation where agent-specific behavior can be integrated without rebuilding standard chat infrastructure from scratch.

Concretely, React provides declarative state management for message streams and UI updates, assistant-ui contributes domain-specific interaction primitives (thread containers, runtime providers, transport abstractions), shadcn/ui-style components provide accessible and reusable UI building blocks, and Tailwind enables rapid and consistent visual composition within the existing modeler interface. This stack lets the implementation focus on collaboration logic (thread state, model context injection, approval flow) rather than reinventing baseline UI concerns such as layout scaffolding, interaction widgets, and component wiring.

This choice is better aligned with the thesis goals because it provides:

(i) **Context-preserving interaction:** Chat and model edits live in the same workspace, reducing mental and operational context switching.

(ii) **Stateful multi-turn support:** Threaded interaction enables iterative correction, clarification, and refinement of model changes.

(iii) **Reduced engineering duplication:** Established UI/component tooling avoids implementing common chat features from first principles, lowering implementation risk and maintenance effort.

Conceptually, this supports the transition from "AI as autocomplete" to "AI as collaborative modeling partner."

6. Human-in-the-Loop

Human-in-the-loop (HITL) is a collaboration concept in which AI generation and human judgment are combined in a controlled decision cycle rather than delegated to fully autonomous execution [31, 7]. In this thesis, HITL means that the agent may propose model transformations, but the user remains the final authority for accepting, rejecting, or refining those transformations.

This concept is necessary because optimization modeling has high semantic sensitivity: an output can be syntactically valid and still be mathematically inappropriate (e.g., incorrect bounds, missing constraints, wrong objective intent). For that reason, correctness cannot rely on generation alone. It must emerge from a joint process in which the agent accelerates drafting and the user validates domain intent.

Operationally, this introduces a loop-type structure: *user request* → *agent proposal* → *validation and presentation* → *user feedback/approval* → *regeneration or commit*. Implementing this reliably requires an orchestration library that can represent explicit states, conditional transitions, tool invocation, and interruption points for human feedback.

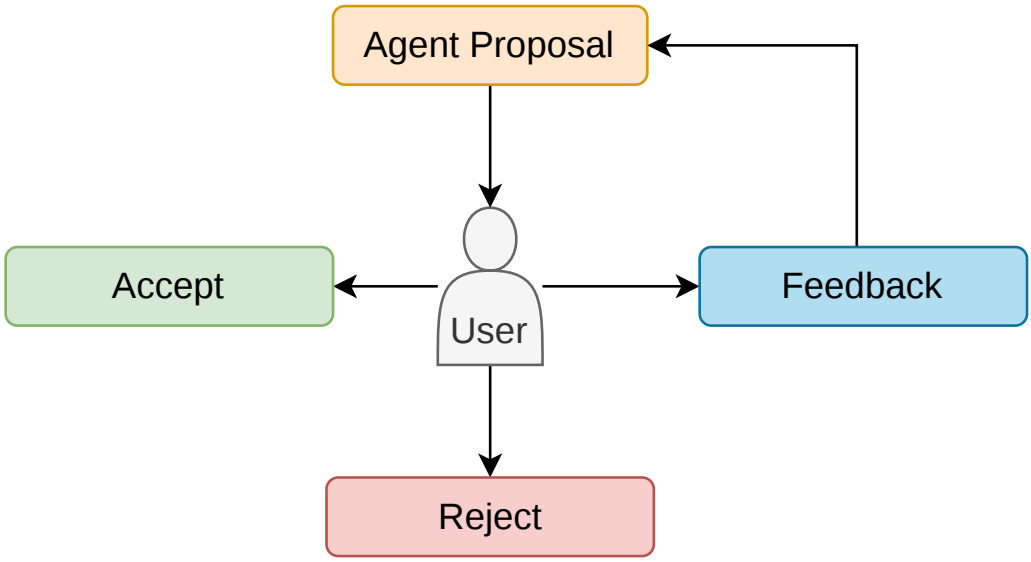

Figure III.3.: Human-in-the-loop decision cycle for AI-assisted optimization modeling.

6.1. Alternatives

At the orchestration-framework level, several alternatives are possible:

- **LangChain [23] Agents:** Fast to prototype for tool-calling scenarios, but less explicit for graph-level state transitions in complex iterative approval flows.
- **LangGraph [24]:** Graph-based orchestration with explicit nodes, edges, conditional routing, and interruption primitives for human approval.
- **n8n [34]:** Strong visual workflow automation and integration ecosystem, but less specialized for fine-grained in-process conversational state management in code-centric agent loops.
- **Microsoft AutoGen / Semantic Kernel style agent frameworks [52]:** Useful for multi-agent and enterprise integration patterns, but often heavier than needed for a single-agent, workspace-centric HITL loop.
- **Custom orchestration from scratch:** Maximum flexibility, but high implementation and maintenance overhead for retry logic, state persistence, interrupt handling, and debugging.

6.2. Choice

This thesis selects **LangGraph** as the orchestration substrate for the HITL cycle. LangGraph models the workflow as an explicit state graph, which directly matches the collaboration loop required by this system: generation, tool execution, user interruption, feedback integration, and conditional continuation.

This choice is preferable for five reasons:

(i) **Native loop representation:** Iterative request-feedback-regenerate cycles can be encoded as first-class graph transitions rather than ad-hoc callback logic.

(ii) **Interrupt-driven human control:** The framework supports explicit pause-and-resume points for user approval, aligning with HITL requirements.

(iii) **Deterministic control flow:** Conditional edges make progression criteria transparent and debuggable, which is critical for thesis-grade system analysis.

(iv) **Tool-calling compatibility:** External actions (e.g., browser interaction endpoints) can be integrated as graph tools without collapsing orchestration clarity.

(v) **Balanced complexity:** Compared to custom orchestration, it reduces engineering overhead while still providing stronger control semantics than purely prompt-driven pipelines.

Accordingly, HITL in this architecture is not an auxiliary safety check but the core execution paradigm: every agent output participates in a supervised loop until either user approval is reached or the proposal is discarded.

7. Backend Services

The backend concept follows a service-oriented architecture that separates orchestration, browser interaction, and modeler integration concerns. In practice, this includes a Python [41]-based orchestration service and a Node.js [35]-based browser-control bridge.

At the infrastructure level, both components are exposed as HTTP servers [50]. An HTTP server is a process that listens for network requests on defined endpoints (routes), executes application logic, and returns structured responses (typically JSON). This is important for the present system because it creates a clean contract between components: the modeler-facing side can call backend capabilities through stable API endpoints without depending on internal implementation details.

Using HTTP-based services offers three practical advantages in this architecture. First, it provides language-agnostic interoperability, allowing Python and JavaScript services to collaborate through a common protocol. Second, it improves modularity by enabling each service (agent orchestration, browser control, validation) to evolve independently. Third, it supports deployment flexibility, since services can run on separate processes or hosts while preserving the same request interface.

7.1. Alternatives

Architecturally, at least three alternatives exist:

- **Monolithic Node backend [39]:** Single runtime for UI integration and browser automation, but reduced access to the Python-first LLM orchestration ecosystem.
- **Monolithic Python backend:** Strong AI ecosystem fit, but less natural for tight integration with existing JavaScript-based browser automation scripts.
- **Microservice decomposition:** Clear separation of concerns, but potentially higher deployment and observability overhead.

7.2. Choice

The selected architecture uses **FastAPI [13] + LangGraph** for agent orchestration and a **Node/Express [11] + Puppeteer** bridge for browser-level actions. This is superior for the current project constraints because:

(i) **FastAPI + LangGraph fit the AI orchestration problem:** LangGraph has strong and mature Python support for state graphs, conditional routing, tool-calling, and interruption-based human feedback. Pairing it with FastAPI provides a lightweight, high-performance HTTP layer for exposing orchestration endpoints in a production-friendly way.

(ii) **Python is the dominant AI integration ecosystem:** Most contemporary AI/LLM tooling, orchestration libraries, and reference implementations are Python-first, which reduces integration friction and accelerates experimentation, debugging, and iteration for agent logic.

(iii) **Node.js + Puppeteer fit browser execution semantics:** Puppeteer is JavaScript-native and directly aligned with browser automation workflows. Because the target environment is a web modeler, using JavaScript for automation is natural: language and runtime conventions match the browser platform and DOM interaction model.

(iv) **JavaScript maps directly to DOM-centric operations:** DOM querying, event triggering, and in-page script evaluation are native web concepts, and JavaScript is the browser's execution language. This makes browser-side mutation and state inspection more direct than cross-language bridging patterns.

(v) **Modular evolution:** Each service can be replaced or scaled independently (e.g., model backend changes without rewriting browser control).

This separation also supports future enhancements such as stronger validation services, improved observability, and safer deployment boundaries.

Chapter IV.

Implementation

This chapter translates the conceptual architecture into the implemented system. It describes how the AI collaborator is integrated into the existing Optimization Platform, how requests move across services, and how generated updates are applied to shared workspaces in a human-in-the-loop workflow. For readers who are new to the platform internals, it is recommended to first review the prior thesis work that established the base collaboration and modeler architecture [33, 6, 32, 30]. Doing so provides the full system context and makes the implementation extensions introduced in this chapter easier to follow.

1. Implementation Overview

1.1. System Components

The implementation is organized around four concrete components:

(i) **Solver-Panel (Angular):** User-facing entry point for authentication, collaboration, and modeler access.

(ii) **Modeler runtime (JavaScript/React integration):** Hosts the visual modeling canvas and embeds the AI chat interface.

(iii) **Agent orchestration service (Python):** Executes stateful LLM workflow logic and human-feedback loops using LangGraph [24].

(iv) **Automation bridge (Node.js):** Controls browser-level interactions and DOM-side workspace updates through Puppeteer [40, 16].

This decomposition preserves existing platform functionality [33, 6, 32, 30] while adding AI collaboration as an orthogonal capability.

1.2. End-to-End Request Path

At runtime, the high-level request path is:

(i) A user invites the AI collaborator into an active editable workspace.

(ii) The frontend mounts and exposes a chat interface inside the modeler context.

(iii) User messages are sent through the chat runtime to the orchestration backend.

(iv) The orchestration service builds context from conversation state and current model state.

(v) If required, tool calls trigger browser-layer operations through HTTP endpoints.

(vi) The resulting model update is applied to the shared workspace and becomes visible to collaborators.

(vii) The user may accept, reject, or refine through an interrupt-driven feedback loop.

1.3. Relevant Files and Project Structure

To make the implementation traceable, this section highlights the primary code locations used in the AI-collaboration pipeline:

- **Solver-Panel integration:** internal modeler component files where invitation and collaboration handoff are implemented.
- **Modeler chat runtime:** modeler files responsible for mounting the assistant UI, injecting model context, and forwarding thread/run calls.
- **Python orchestration:** LangGraph workflow definition, tool binding, and prompt-control logic.
- **Node automation bridge:** HTTP endpoints and Puppeteer methods for browser session initialization and model DOM read/write operations.

Because the repository is a multi-module workspace, a complete structure listing is provided in the appendix for reproducibility and navigation. The detailed tree and file-role mapping are documented in Chapter A.

2. Frontend Integration

For detailed API endpoint documentation related to this section (invitation, runtime communication, and service calls), see Chapter C.

2.1. Agent Invitation

The integration extends the existing collaboration workflow by adding an "Invite AI Agent" action in the internal modeler view, implemented through the `inviteAI()` method in `internal-modeler.component.ts`. This method is the concrete frontend entry point for agent invocation.

Operationally, `inviteAI()` performs the following sequence:

(i) Ensure that a collaboration context exists; if not, create one through the existing master-user collaboration builder.

(ii) Retrieve the workspace edit link from the master collaboration object (the write-capable collaboration URL).

(iii) Send an HTTP request to the automation bridge endpoint:

```
POST /puppet
```

with JSON payload:

```
{"link": <writeUrl>}
```

(iv) Update frontend UI state to expose the AI collaborator controls after successful invocation.

On the backend side, this endpoint is implemented in the Node/Express bridge service (`AI-Collaborator/puppeteer-server/index.js`) as `POST /puppet`. When called, the endpoint invokes a Puppeteer initialization routine that launches a server-side Chrome session. The automation routine logs the agent into the platform using dedicated credentials, then navigates to the provided collaborative write URL. This browser session serves as the AI agent's execution environment, enabling the agent to join the current collaboration session as an editing participant and interact with the same live modeler state as human users.

The invitation path remains role-aware because it reuses the platform's existing edit/read collaboration semantics. Therefore, no parallel permission model is introduced; the AI participant is integrated through the same collaboration boundary already used for human users.

2.2. Embedded Chat Runtime

The chat interface is mounted directly in the modeler container, not as an external application. The implementation uses assistant-ui with a React-based modal/thread runtime [19], so user interaction remains co-located with model editing.

Communication between assistant-ui and LangGraph is implemented through the LangGraph runtime adapter in the frontend. Concretely, the runtime defines three core operations: `create`, `load`, and `stream`. During `create`, the frontend requests a new thread from the LangGraph API and stores the returned external thread identifier in the assistant-ui runtime state. During `load`, the frontend retrieves the thread state from LangGraph and reconstructs the local conversation view (messages and pending interrupts). This ensures that chat state remains persistent across UI reloads and modal reopen events.

During active interaction, assistant-ui invokes the `stream` operation for each user turn. The frontend sends the message payload (plus runtime command metadata) to the LangGraph run-stream endpoint and subscribes to incremental response events. These streamed events are rendered token-by-token in the assistant-ui thread, which provides immediate user feedback and preserves conversational continuity. In parallel, command responses from LangGraph (e.g., interrupt/resume requirements in the human-in-the-loop

flow) are surfaced in the same thread UI so that users can explicitly approve, reject, or refine the current proposal.

At the protocol level, assistant-ui is therefore responsible for interaction orchestration in the browser (thread lifecycle, rendering, user actions), while LangGraph is responsible for stateful agent execution on the backend (graph transitions, tool calls, and interrupt logic). The integration boundary is a thread-oriented API contract: assistant-ui sends user turns and commands, and LangGraph returns streamed model outputs plus state updates.

2.3. Model Context Injection

For each conversational turn, the frontend retrieves the current state of the active modeler instance and injects it into the request context before forwarding the user message. This mechanism is implemented in the chat runtime path by querying the modeler state (HTML/instance representation) and prepending it as a system-level context message for the agent run.

The injected context serves two implementation goals. First, it grounds the model in the *current* workspace snapshot, including already defined variables, constraints, objectives, and structural nesting. Second, it provides enough local state to interpret referential instructions such as "modify the existing capacity constraint," "add this variable to the objective," or "update the bound we added earlier." Without explicit context injection, such instructions would be ambiguous or incorrectly resolved.

From a data-flow perspective, each user turn is transformed into a composite payload:

(i) current workspace/model representation (system context),

(ii) conversational history (thread messages),

(iii) current user instruction (new turn),

(iv) optional runtime command metadata (e.g., interrupt resume input).

This design improves consistency across iterative runs, because the agent does not rely on implicit memory alone; it receives an explicit state snapshot at invocation time. In practice, this reduces context drift and helps keep generated updates aligned with the latest model state, which is particularly important in collaborative sessions where the workspace can change between turns.

3. Agent Orchestration Service

3.1. LangGraph Workflow Realization

The orchestration backend is implemented as a state graph with explicit nodes and conditional transitions [24]. In API terms, the frontend and backend communicate through LangGraph's thread-and-run abstraction introduced in the embedded chat subsection:

- **Thread:** persistent conversation/state container for one chat session.
- **create:** initializes a new thread and returns an external thread identifier.
- **load:** retrieves stored thread state (messages and pending interrupts).
- **stream:** starts a run on a thread and returns incremental output events.
- **command:** runtime control input, used for actions such as interrupt resume.

Operationally, assistant-ui uses these API primitives to drive LangGraph execution: each user turn is sent through a streamed run bound to a thread ID; LangGraph then executes the state graph, emits partial outputs, and persists updated state back to that same thread.

At graph level, the implemented flow includes:

- an LLM node for proposal generation,
- validation and repair nodes that gate model updates,
- an apply node that writes changes to the live workspace,
- and a human-feedback node for interrupt/resume control.

This graph-level structure directly realizes the intended HITL execution loop:

$$\text{request} \rightarrow \text{generation} \rightarrow \text{tool/action} \rightarrow \text{feedback} \rightarrow (\text{regenerate or terminate})$$

More concretely, LangGraph executes this loop by evaluating conditional edges after each node execution. If the LLM response requires a tool call, control moves to the tool node; if human approval is required, an interrupt state is emitted and exposed to the frontend. The frontend then sends a command (resume payload), and graph execution continues from the interrupt point instead of restarting from scratch. This resume semantics is the key mechanism that turns a normal chat pipeline into a stateful human-in-the-loop workflow.

Compared to ad-hoc request handlers, this implementation provides explicit state transitions, observable control flow, deterministic routing conditions, and durable conversational state across turns.

3.2. Graph States

The implemented orchestration can be described as a set of explicit graph states, where each state has a narrowly defined responsibility in the request lifecycle.

LLM State (Generation State). This state is responsible for proposal generation. It receives the composite context (thread history, current model snapshot, and current user instruction), applies system-level constraints, and produces either:

- a candidate model update, or
- a tool-call request when external environment interaction is required.

In other words, this state determines *what* should happen next.

Execution State (Validation/Apply). These states execute external actions through HTTP calls to the automation bridge, such as validating candidate markup, reading current model content, or applying updates to the active workspace. The results of these calls are fed back into graph state so the next LLM step can reason over fresh environment results. This state therefore implements *environment-grounded execution* rather than pure text generation.

HITL State (Approval/Feedback State). This state materializes human-in-the-loop control through interrupt and resume semantics. When user confirmation is required, the graph emits an interrupt that is surfaced to the frontend. The user can then approve, reject, or provide refinement feedback; the returned command resumes graph execution from the suspended state. This keeps user authority inside the orchestration loop rather than outside it.

Validation State (Reliability State). A dedicated validation state has been integrated into the implemented graph. This state executes before the final commit and verifies that AI-generated output conforms to modeler constraints and schema expectations. Instead of introducing an ad-hoc validator, the implementation reuses the validation logic already present in the modeler stack (i.e., existing modeler validators and checks) by calling the automation bridge `POST /validate-html` endpoint. This provides a deterministic acceptance gate:

$$\text{generated output} \rightarrow \text{modeler validator checks} \rightarrow (\text{accept} \mid \text{return errors for refinement})$$

and would further reduce the risk of structurally valid but semantically inconsistent updates.

3.3. External Actions and Bridge Calls

The orchestration service triggers external actions by calling the Node automation bridge over HTTP. Instead of a separate tool node, these actions are represented as explicit graph states that call bridge endpoints and feed responses back into the LangGraph state.

The relevant runtime actions are:

- **`get_current_html()`**: Retrieves the current model representation from the active browser-side modeler instance via `GET /get-html`. This is used when the agent needs fresh environment state before generating or refining an update.
- **`validate_html(new_html)`**: Validates candidate markup using the modeler validators exposed through `POST /validate-html`. Validation errors are routed into the repair path.
- **`update_html(new_html)`**: Applies generated model content back to the live modeler DOM through `POST /update-html`. This is the execution step that materializes the agent proposal in the shared workspace.

In addition, development/testing helper tools (e.g., arithmetic helpers used during graph experiments) can be bound in the same mechanism, but they are not part of the production modeling flow.

These tools call the Node automation bridge via HTTP, which keeps environment execution separate from LLM reasoning logic.

This separation is architecturally important: the Python service decides *what* change to perform, while the automation bridge performs *how* the change is enacted in the browser session.

3.4. Prompt and Constraint Strategy

The implemented prompting strategy uses a strong *system prompt* as a behavioral contract for the agent. In this context, the system prompt is the highest-priority instruction layer passed to the model at run time; it defines the agent role, task boundaries, expected output format, and non-negotiable constraints. User messages are interpreted within this envelope rather than in isolation. For the full prompt specification used in this thesis implementation, see Chapter D.

For this thesis, the system prompt is designed around the optimization modeler's domain-specific markup language. It explicitly specifies:

- required structural organization (root tags, mandatory blocks, nesting rules),
- allowed elements and attributes for variables, objectives, constraints, loops, and script blocks,
- and validation-critical prohibitions (forbidden nesting patterns and structurally invalid combinations).

This design reduces under-specified generation and keeps outputs aligned with the modeler's parser and execution expectations.

The prompt also helps in three practical ways. First, it improves **format reliability** by reducing malformed outputs. Second, it improves **semantic alignment** by reminding the model that it acts as a mathematical modeling assistant, not a generic chatbot. Third, it improves **repairability**: when validation errors occur, the same constraint framework can be reused to guide the model toward corrected output in the next iteration.

In addition to core instructions, the implementation can include *example-based prompting*. Terminologically, this is often referred to as **one-shot** (one example) or **few-shot** (multiple examples) prompting. This is not model training in the machine-learning sense; instead, it is in-context conditioning at inference time. The examples demonstrate target input-output behavior (e.g., natural-language request to valid markup transformation), which helps the model imitate the desired structure and reduce ambiguity in difficult cases.

Overall, the prompt layer acts as a soft specification interface between natural-language intent and executable model transformations, complementing downstream validation and human-in-the-loop supervision.

3.5. Validation Gate Before Model Application

Before any AI-generated optimization model update is committed to the collaborative workspace, it should pass a validation gate based on the modeler package's existing validators/checks. The purpose of this gate is to ensure that generated output is not only well-formed text, but structurally and semantically compatible with the modeler's expected representation.

Conceptually, the validation pipeline is:

$$\text{LLM output} \rightarrow \text{modeler validators} \rightarrow (\text{accept and apply} \mid \text{reject and refine})$$

In this design, validation is executed before DOM mutation. If the validator layer accepts the generated artifact, the update is forwarded to the automation execution path and applied to the active modeler instance. If validation fails, the error diagnostics are returned to the orchestration loop and used as structured feedback for the next generation attempt. This keeps invalid or partially inconsistent updates from propagating to collaborators.

Reusing modeler-native validators has two implementation advantages. First, it aligns AI output quality checks with the same rule set already used by the platform, avoiding divergence between AI and manual editing paths. Second, it creates a deterministic acceptance criterion that can be audited and improved incrementally as the validator suite evolves.

4. Browser Automation Execution Layer

4.1. Session Initialization

The Node.js automation bridge exposes HTTP endpoints for browser lifecycle actions. Initialization starts when the frontend invitation flow calls:

```
POST /puppet
```

with payload containing the collaborative write URL. In the bridge service, this endpoint delegates to the Puppeteer bootstrap method (`puppetInit(link)`), which performs the complete startup sequence.

Concretely, initialization executes the following steps:

(i) **Browser launch:** start a Puppeteer-controlled Chrome instance on the server side and create a new page context.

(ii) **Viewport/session setup:** configure runtime viewport and browser page state used for subsequent modeler interaction.

(iii) **Authentication flow:** navigate to the platform login route and submit AI-account credentials through scripted form interactions.

(iv) **Workspace attach:** after successful login, navigate to the received collaboration write URL.

(v) **Persistent page binding:** store the initialized page handle for later read/write calls (model retrieval and model update endpoints).

After this sequence, the automation layer is attached to the same live collaborative modeler session as human users. Subsequent tool calls can therefore operate on the active workspace state without reinitializing browser context on every request.

4.2. Workspace Read/Write Operations

After initialization, the runtime follows a concrete execution path for each user prompt. The main steps are as follows:

(i) **Prompt-to-tool transition:**
The user prompt is processed in the LangGraph loop. When the LLM determines that a model mutation is required, it emits a call to the bound `update_html(new_html)` tool.

(ii) **Tool invocation and endpoint call:**
The Python tool function `update_html` sends a

```
POST /update-html
```

request to the Node automation bridge with the following payload:

```
{"newHTML": <generated-model-markup>}
```

This endpoint is handled in `AI-Collaborator/puppeteer-server/index.js`, where the request is forwarded to `puppetEditModel(newHTML)`.

(iii) **Browser-side model update:**
Inside `puppetEditModel` (`AI-Collaborator/puppeteer-server/puppet.js`), Puppeteer calls a modeler-native update hook:

```
window.Modeler.updateModelViewHTML(newHTML)
```

This operation replaces the content of the active model instance through the modeler runtime API rather than direct DOM mutation.

(iv) **Propagation to collaborators:**
Because the mutation occurs directly in the live collaborative modeler DOM, the platform's existing synchronization channel propagates the resulting workspace change to connected participants. Thus, the agent update appears in the same shared session context as human edits.

Additionally, read operations are available through `get_current_html()` (Python action) and `GET /get-html` (Node bridge), enabling the agent to fetch the latest workspace state before subsequent generation cycles. Validation is executed through `POST /validate-html`, which delegates to `window.Modeler.validateHTML(newHTML)` inside the Puppeteer session.

5. Portability and Adaptation to Other Platforms

While the implementation described in the preceding sections targets the Optimization Platform's modeler, the architecture was designed with explicit separation between *platform-agnostic infrastructure* and *platform-specific integration points*. This section identifies the concrete adaptation boundaries and explains how the same architecture can be ported to a different web application with minimal structural changes.

5.1. Architectural Separation Principle

The AI-collaboration pipeline is organized into three layers, each with a different scope of platform coupling:

(i) **Orchestration layer (Python/LangGraph):** Stateful workflow execution, LLM invocation, human-in-the-loop control, and conditional graph routing. This layer is fully platform-agnostic; it communicates with the environment exclusively through HTTP calls to the automation bridge. No platform-specific logic resides here except for the system prompt content.

(ii) **Automation bridge (Node.js/Puppeteer):** Browser session management and HTTP endpoint exposure. The bridge is split into a generic session initialization component and platform-specific read/write/validate operations.

(iii) **Frontend integration (chat runtime):** The assistant-ui chat runtime and its LangGraph adapter. The chat widget itself is framework-independent (React-based, embeddable); only the model-context injection logic is coupled to the host application's state representation.

This layering means that porting the architecture to a different platform requires changes only at well-defined integration points, without restructuring the orchestration graph, the chat runtime protocol, or the bridge server skeleton.

5.2. Automation Bridge: Generic vs. Platform-Specific Endpoints

In the implemented bridge service (`AI-Collaborator/puppeteer-server/`), the code explicitly separates generic and platform-specific concerns. The session initialization endpoint (`POST /puppet`) and the Puppeteer bootstrap routine (`puppetInit`) are designed to work with any web application that has a login page. Authentication details are externalized as named constants:

- `PLATFORM_LOGIN_URL`: the URL of the target application's login page.
- `PLATFORM_EMAIL_LOCATOR`, `PLATFORM_PASSWORD_LOCATOR`, `PLATFORM_SUBMIT_BUTTON_LOCATOR`: CSS selectors for the authentication form elements.
- `AI_USER_EMAIL`, `AI_USER_PASSWORD`: dedicated agent credentials.

To adapt session initialization to a different platform, a developer replaces these constants with values matching the target application's login page structure. The initialization flow itself (launch browser, navigate, authenticate, attach to workspace URL) remains unchanged.

The platform-specific endpoints (`POST /update-html`, `GET /get-html`, `POST /validate-html`) and their corresponding Puppeteer functions (`puppetEditModel`, `puppetGetModel`, `puppetValidateModel`) are explicitly documented as platform-dependent. These functions assume that the target application exposes certain methods on a global JavaScript object (in this case, `window.Modeler`) that the Puppeteer session can call via `page.evaluate()`. For a different platform, a developer would:

(i) Define equivalent read, write, and validate operations for the target application's DOM or API surface.

(ii) Implement corresponding `page.evaluate()` calls that invoke the target application's exposed hooks.

(iii) If the target application does not expose such hooks natively, implement thin adapter methods in the application's frontend code that bridge between the Puppeteer call and the internal application state.

The bridge endpoint signatures (HTTP verbs, payload shapes, response formats) can remain identical, which means the orchestration layer requires no changes when the bridge internals are swapped.

5.3. Orchestration Layer: Prompt as the Domain Interface

The LangGraph workflow graph (node topology, conditional edges, interrupt/resume logic) is entirely domain-independent. The graph structure shown in the preceding sections – generation, validation, repair, apply, and human-feedback states – applies to any scenario where an LLM proposes structured changes to an editable workspace and a human reviews them before commitment.

The only component in the orchestration layer that carries platform-specific semantics is the **system prompt**. In the current implementation, the system prompt encodes the grammar rules, structural constraints, and output format expectations of the optimization modeler's markup language. For a different platform, the system prompt would be replaced with an equivalent specification of the target domain's representation language or editing conventions.

In practice, adapting the orchestration layer to a new domain requires:

- writing a new system prompt that defines the target output format, structural rules, and domain role,
- and optionally adjusting the validation/repair prompts (`system_prompt_fix_html`, `system_prompt_update_html`) to reference the new domain's error semantics.

No changes to the graph topology, state schema, or routing logic are necessary.

5.4. Frontend Integration: Embeddable Chat Runtime

The assistant-ui chat widget communicates with the LangGraph backend through a standardized thread-and-run API contract (create, load, stream). This contract is not specific to any host application; it depends only on the LangGraph server endpoint and a graph identifier. Therefore, embedding the chat interface into a different web application requires:

(i) mounting the assistant-ui React component in the target application's UI container,

(ii) configuring the LangGraph client URL and graph ID in the chat API module,

(iii) and implementing a context-injection hook that retrieves the current application state and prepends it to each user message.

The context-injection hook is the primary platform-specific adaptation point on the frontend side. In the current implementation, it queries the modeler's HTML representation; in a different application, it would query whatever state representation the target platform uses (e.g., a JSON document, a canvas state, or a code editor buffer).

5.5. Summary of Adaptation Points

Table IV.1 summarizes the concrete changes required to port the architecture to a different web platform.

Table IV.1.: Platform-specific adaptation points for porting the AI collaborator architecture.

Layer	**What to change**	**What remains unchanged**
Automation bridge (init)	Login URL, form selectors, agent credentials	Browser launch, viewport setup, workspace-attach flow
Automation bridge (ops)	`page.evaluate()` hooks for read/write/validate	HTTP endpoint signatures, bridge server skeleton
Orchestration (Python)	System prompt, repair/update prompt templates	Graph topology, state schema, routing, HITL logic
Frontend (chat)	Context-injection hook, UI mount point	assistant-ui runtime, LangGraph adapter, thread API

Because the platform-specific surface is limited to configuration constants, a small set of browser-side hooks, and prompt content, the overall architecture can be reused across web applications without reimplementing the orchestration, chat runtime, or automation infrastructure.

Chapter V.

Evaluation

1. Evaluation Goals

This chapter evaluates whether the implemented AI collaborator addresses the research questions formulated in the introduction:

- Can an integrated, stateful, and supervised agent architecture reduce modeling effort for routine and repetitive tasks?
- Can such an architecture lower entry barriers for users with limited optimization expertise?
- Can it improve the efficiency and quality of collaborative MILP model development?

To answer these questions, we evaluate the system along three dimensions: efficiency, accessibility, and output quality under human supervision.

2. Experimental Setup

2.1. System Configuration

All experiments are conducted on the integrated prototype described in this thesis, using the same execution pipeline for all tasks and participants. The LLM used for generation is accessed via OpenRouter and configured with the model identifier `openai/gpt-oss-120b:free`. Each run uses the same system-prompt contract and identical tool availability to ensure configuration consistency across manual-vs-AI comparisons.

The runtime environment consists of three local service endpoints: (i) the modeler web client session, (ii) a LangGraph-backed orchestration runtime for threaded inference and interrupt handling [24], and (iii) a browser-automation service using Puppeteer for environment-side model read/write operations [40]. AI-generated updates are applied directly to the live collaborative workspace, and all measurements are collected from this end-to-end setup rather than from isolated offline prompts.

2.2. Task Design

We use a set of representative MILP modeling tasks with increasing complexity (small, medium, and large). Each task requires users to create or modify variables, objectives, and constraints in the modeler. For each task, two execution modes are compared:

(i) **Manual mode:** modeling only through direct UI interaction.

(ii) **AI-assisted mode:** modeling with the first-class AI collaborator through natural-language requests and iterative feedback.

2.3. Participants and Procedure

Participants are grouped by experience level (e.g., beginner/intermediate/advanced with optimization modeling). Each participant solves the same task set in both modes. To reduce learning effects, task order should be counter-balanced across participants.

For each run, we log completion time, interaction effort, number of correction cycles, and final model validity. The same acceptance criteria are used for manual and AI-assisted runs.

3. Metrics

3.1. Efficiency Metrics

- **Task Completion Time (TCT):** end-to-end time from task start to accepted final model.
- **Modeling Speed:** number of completed modeling operations per minute.
- **Speedup Factor:**

$$\text{Speedup} = \frac{T_{\text{manual}}}{T_{\text{AI}}}$$

  where values > 1 indicate faster completion with AI assistance.

3.2. Effort and Accessibility Metrics

- **Interaction Count:** number of UI operations (manual mode) versus number of prompts/feedback turns (AI mode).
- **Correction Iterations:** number of refinement cycles until user acceptance.
- **Onboarding Performance:** task success rate and completion time for low-experience users.

3.3. Quality and Reliability Metrics

- **Syntactic Validity Rate:** fraction of generated updates passing structural checks.
- **Accepted Suggestion Ratio:** accepted proposals divided by total AI proposals.
- **Final Model Quality:** whether resulting models satisfy task requirements and solver feasibility conditions.

4. Results by Research Question

4.1. RQ1: Reduction of Modeling Effort

To answer RQ1, we compare manual and AI-assisted completion times and interaction counts for routine and repetitive modeling tasks.

Table V.1.: Manual vs AI-assisted efficiency comparison (fill with measured values).

Task	Manual Time (min)	AI Time (min)	Speedup
Small task	–	–	–
Medium task	–	–	–
Large task	–	–	–

Interpretation criterion: if AI-assisted runs show consistently lower completion time and reduced interaction overhead, RQ1 is supported.

4.2. RQ2: Lowering Entry Barriers

To answer RQ2, we focus on participants with limited optimization/modeler experience and compare:

- task completion rate,
- completion time,
- and number of blocked situations (periods where users cannot proceed without support).

Table V.2.: Accessibility indicators for low-experience users (fill with measured values).

Metric	Manual	AI-Assisted	Improvement
Completion rate (%)	–	–	–
Avg. completion time (min)	–	–	–
Blocked situations per task	–	–	–

Interpretation criterion: improved completion outcomes and fewer blocked situations in AI-assisted mode support RQ2.

4.3. RQ3: Efficiency and Quality in Collaborative Development

To answer RQ3, we evaluate collaborative sessions where users and the AI agent co-develop a model through iterative proposal-review cycles.

Table V.3.: Collaborative quality and supervision metrics (fill with measured values).

Metric	Value	Interpretation
Syntactic validity rate (%)	–	Structural reliability of generated updates
Accepted suggestion ratio (%)	–	Practical usefulness of proposals
Avg. correction iterations	–	Refinement overhead under HITL
Final feasible models (%)	–	End-state model quality

Interpretation criterion: high validity and feasibility with manageable correction loops support RQ3, showing that collaboration quality improves without removing user control.

5. Threats to Validity

5.1. Internal Validity

Learning effects and prompt-quality variance may influence measured performance. Counter-balanced task ordering and standardized task instructions mitigate these effects.

5.2. External Validity

Results from prototype tasks and a limited participant set may not generalize to all optimization domains. Future studies should include broader industrial case studies and larger user cohorts.

5.3. Construct Validity

Time-based metrics alone cannot capture all benefits of collaboration quality. Therefore, efficiency metrics are complemented by validity, acceptance, and refinement metrics.

6. Summary

This evaluation framework directly addresses all three research questions by combining quantitative efficiency comparisons (manual versus AI-assisted), accessibility indicators for less-experienced users, and quality/reliability metrics for collaborative HITL modeling. The resulting evidence determines whether the implemented first-class AI collaborator improves modeling speed, reduces entry barriers, and increases collaborative model-development quality.

Chapter VI.

Conclusion

1. Summary

This thesis investigated how an AI agent can be integrated as a first-class collaborator in a web-based optimization platform for Mixed-Integer Linear Programming (MILP). The core objective was not to add a detached assistant, but to embed an execution-capable agent directly into the existing collaborative modeling workflow.

The implemented architecture combines: (i) in-modeler chat interaction, (ii) stateful LangGraph-based orchestration, (iii) a browser automation bridge using Puppeteer for environment-side execution, and (iv) human-in-the-loop supervision with iterative feedback. This design allows the agent to operate on the current shared workspace state, generate context-aware model updates, and apply validated changes through the same collaboration channel used by human participants.

With respect to the research questions introduced in this thesis, the work provides the following contributions:

- **Reduction of modeling effort:** The architecture operationalizes natural-language-driven model editing and establishes an evaluation framework for measuring manual versus AI-assisted modeling speed.
- **Lowering entry barriers:** The integrated assistant provides on-demand support inside the existing modeling interface, reducing dependence on deep prior platform expertise.
- **Improved collaborative quality under supervision:** The system formalizes a stateful proposal-review loop where users retain final authority over acceptance, rejection, and refinement of generated updates.

Overall, the thesis demonstrates that AI support in optimization modeling is most effective when it is stateful, environment-aware, and explicitly supervised, rather than purely autonomous or prompt-only.

2. Limitations and Future Work

Although the prototype demonstrates feasibility and end-to-end integration, several limitations remain and motivate future work.

2.1. Current Limitations

- The implementation currently prioritizes research prototyping and architectural integration over production hardening.
- Evaluation results depend on the selected task set and participant distribution; broader empirical studies are required for stronger generalization claims.
- Reliability can be further improved by expanding and standardizing validator coverage across all model transformation cases.

2.2. Future Directions

- **Extended validation pipeline:** Integrate modeler-native validators as a stricter pre-commit gate across all update paths, with richer error diagnostics for iterative correction.
- **Larger-scale evaluation:** Conduct broader user studies with diverse optimization domains and statistically grounded comparisons of efficiency, accessibility, and quality metrics.
- **Operational hardening:** Improve observability, service robustness, and deployment standardization for long-running collaborative sessions.
- **Interaction refinement:** Add richer UI feedback for agent state transitions (generation, validation, waiting-for-approval) to further improve transparency and trust.

In conclusion, this work establishes a practical and extensible foundation for human-AI co-modeling in optimization platforms and shows that first-class collaborative agents are a viable direction for improving both productivity and accessibility in technical modeling workflows.

Bibliography

[1] *AI features — overleaf.com*. https://www.overleaf.com/about/ai-features. [Accessed 26-04-2026].

[2] Anthropic. *Claude Code Agent: A Code Assistant for Claude*. 2025. URL: https://code.claude.com/docs/en/overview.

[3] *antigravity.google*. https://antigravity.google/. [Accessed 26-04-2026].

[4] *API Overview — developers.openai.com*. https://developers.openai.com/api/reference/overview. [Accessed 04-05-2026].

[5] *API Platform — openai.com*. https://openai.com/api/. [Accessed 04-05-2026].

[6] Deni Begaj. *A multi-user platform for managingand solving Mixed Integer Linear Optimization problems*. Master Thesis at Saarland University. 2020.

[7] Filippos Bellos et al. *Towards Effective Human-in-the-Loop Assistive AI Agents*. 2025. arXiv: 2507.18374 [cs.CV]. URL: https://arxiv.org/abs/2507.18374.

[8] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. 2021. arXiv: 2107.03374 [cs.LG]. URL: https://arxiv.org/abs/2107.03374.

[9] *Cursor · Agent — cursor.com*. https://cursor.com/product. [Accessed 26-04-2026].

[10] *Document Object Model (DOM) - Web APIs | MDN — developer.mozilla.org*. https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model. [Accessed 16-01-2026].

[11] *Express - Node.js web application framework — expressjs.com*. https://expressjs.com/. [Accessed 04-05-2026].

[12] *Fast and reliable end-to-end testing for modern web apps | Playwright Python — playwright.dev*. https://playwright.dev/python/. [Accessed 04-05-2026].

[13] *FastAPI — fastapi.tiangolo.com*. https://fastapi.tiangolo.com/. [Accessed 04-05-2026].

[14] Ali Forootani. *A Survey on Mathematical Reasoning and Optimization with Large Language Models*. 2025. arXiv: 2503.17726 [cs.AI]. URL: https://arxiv.org/abs/2503.17726.

[15] Google. *Gemini CLI: A Command-Line Interface for Gemini*. 2025. URL: https://geminicli.com/.

[16] *Home — chromium.org*. https://www.chromium.org/chromium-projects/. [Accessed 16-01-2026].

[17] Xuhan Huang et al. *LLMs for Mathematical Modeling: Towards Bridging the Gap between Natural and Mathematical Languages*. 2025. arXiv: 2405.13144 [cs.AI]. URL: https://arxiv.org/abs/2405.13144.

[18] *Integer programming — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Integer%20programming&oldid=1325150256. 2026.

[19] *Introduction — assistant-ui.com*. https://www.assistant-ui.com/docs. [Accessed 26-04-2026].

[20] Caigao Jiang et al. *LLMOPT: Learning to Define and Solve General Optimization Problems from Scratch*. 2025. arXiv: 2410.13213 [cs.AI]. URL: https://arxiv.org/abs/2410.13213.

[21] Juyong Jiang et al. "A Survey on Large Language Models for Code Generation". In: *ACM Transactions on Software Engineering and Methodology* 35.2 (Jan. 2026), pp. 1–72. ISSN: 1557-7392. DOI: 10.1145/3747588. URL: http://dx.doi.org/10.1145/3747588.

[22] Rajendra Kunwar and HP Sapkota. "An introduction to linear programming problems with some real-life applications". In: *European Journal of Mathematics and Statistics* 3.2 (2022), pp. 21–27.

[23] *LangChain overview - Docs by LangChain — docs.langchain.com*. https://docs.langchain.com/oss/python/langchain/overview. [Accessed 04-05-2026].

[24] *LangGraph — langchain.com*. https://www.langchain.com/langgraph. [Accessed 16-01-2026].

[25] *Large language model - Wikipedia — en.wikipedia.org*. https://en.wikipedia.org/wiki/Large_language_model. [Accessed 16-01-2026].

[26] *Linear programming — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Linear%20programming&oldid=1329325329. 2026.

[27] *LiteLLM — litellm.ai*. https://www.litellm.ai/. [Accessed 04-05-2026].

[28] Jiawei Liu et al. *Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation*. 2023. arXiv: 2305.01210 [cs.SE]. URL: https://arxiv.org/abs/2305.01210.

[29] *Microsoft 365 Copilot | KI-Produktivitätswerkzeuge für die Arbeit — microsoft.com*. https://www.microsoft.com/de-de/microsoft-365-copilot. [Accessed 26-04-2026].

[30] Maksim Moor. *From No-Code to Low-Code: Augmenting Visual Modeling for Mixed-Integer Optimization*. Bachelor Thesis at Saarland University. 2024.

[31] Hussein Mozannar et al. *Magentic-UI: Towards Human-in-the-loop Agentic Systems*. 2025. arXiv: 2507.22358 [cs.AI]. URL: https://arxiv.org/abs/2507.22358.

[32] Marco Mrsic. *An interface to model and solve Mixed-Integer Linear Programs*. Bachelor Thesis at Saarland University. 2020.

[33] Marco Mrsic. *Real-Time Collaborative Optimization Platform*. Master Thesis at Saarland University. 2023.

[34] *n8n.io - AI workflow automation platform — n8n.io*. https://n8n.io/. [Accessed 04-05-2026].

[35] *Node.js — Run JavaScript Everywhere — nodejs.org*. https://nodejs.org/en. [Accessed 04-05-2026].

[36] *Ollama — ollama.com*. https://ollama.com/. [Accessed 04-05-2026].

[37] OpenAI. *Copilot: A Code Assistant for Copilot*. 2025. URL: https://github.com/features/copilot.

[38] *OpenRouter — openrouter.ai*. https://openrouter.ai/. [Accessed 04-05-2026].

[39] Ian Smalley Phill Powell. *What is Monolithic Architecture? | IBM — ibm.com*. https://www.ibm.com/think/topics/monolithic-architecture. [Accessed 04-05-2026].

[40] *Puppeteer | Puppeteer — pptr.dev*. https://pptr.dev/. [Accessed 16-01-2026].

[41] *python.org*. https://www.python.org/. [Accessed 04-05-2026].

[42] *React — react.dev*. https://react.dev/. [Accessed 04-05-2026].

[43] *Selenium — selenium.dev*. https://www.selenium.dev/. [Accessed 04-05-2026].

[44] shadcn. *The Foundation for your Design System - shadcn/ui — ui.shadcn.com*. https://ui.shadcn.com/. [Accessed 04-05-2026].

[45] *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML. — tailwindcss.com*. https://tailwindcss.com/. [Accessed 04-05-2026].

[46] *The AI for Problem Solvers | Claude by Anthropic — claude.com*. https://claude.com/product/overview. [Accessed 04-05-2026].

[47] *v0 by Vercel - Build Agents, Apps, and Websites with AI — v0.app*. https://v0.app/. [Accessed 26-04-2026].

[48] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

[49] *Visual Studio Code - The open source AI code editor | Your home for multi-agent development — code.visualstudio.com*. https://code.visualstudio.com/. [Accessed 26-04-2026].

[50] *What is a web server? - Learn web development | MDN — developer.mozilla.org*. https://developer.mozilla.org/en-{U}{S}/docs/{L}earn_web_development/{H}owto/{W}eb_mechanics/{W}hat_is_a_web_server. [Accessed 04-05-2026].

[51] Google Workspace. *AI Tools for Business | Google Workspace — workspace.google.com*. http://workspace.google.com/solutions/ai/. [Accessed 26-04-2026].

[52] Qingyun Wu et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. 2023. arXiv: 2308.08155 [cs.AI]. URL: https://arxiv.org/abs/2308.08155.

[53] Jian Yang et al. *From Code Foundation Models to Agents and Applications: A Comprehensive Survey and Practical Guide to Code Intelligence*. 2025. arXiv: 2511.18538 [cs.SE]. URL: https://arxiv.org/abs/2511.18538.

[54] Jihai Zhang et al. *Solving General Natural-Language-Description Optimization Problems with Large Language Models*. 2024. arXiv: 2407.07924 [math.OC]. URL: https://arxiv.org/abs/2407.07924.

[55] Shengyu Zhang et al. *Instruction Tuning for Large Language Models: A Survey*. 2025. arXiv: 2308.10792 [cs.CL]. URL: https://arxiv.org/abs/2308.10792.

[56] Wayne Xin Zhao et al. *A Survey of Large Language Models*. 2026. arXiv: 2303.18223 [cs.CL]. URL: https://arxiv.org/abs/2303.18223.

Appendix A.

Implementation File Structure

This appendix provides an implementation-oriented file map for the AI collaborator architecture described in the main chapters. The listing focuses on the modules that are directly relevant for invitation flow, chat runtime, orchestration, and browser automation.

1. High-Level Repository Layout

```
optimization-platform/
 Solver-Panel/
    src/app/functions/internal-modeler/
        internal-modeler.component.ts
        internal-modeler.component.html
        collaboration/
            collaboration-master-user.ts
            collaboration-user-builder.ts
            collaboration.contracts.ts
 Modeler/
    src/
       modeler.js
       mountAIAgent.jsx
       ModalAIAgent.tsx
    components/
       assistant-ui/
           thread.tsx
           thread-list.tsx
    lib/
        chatApi.ts
 AI-Collaborator/
     puppeteer-server/
        index.js
        puppet.js
     server/
         src/agent.py
         langgraph.json
         pyproject.toml
```

2. File Responsibilities

- **`internal-modeler.component.ts/.html`**: Adds and wires the "Invite AI Agent" action (`inviteAI()`) and collaboration link handoff from frontend to the automation bridge via `POST /puppet`.
- **`collaboration-master-user.ts`**: Provides the `writeUrl` used by `inviteAI()` to pass the writable collaboration link to the bridge.
- **`modeler.js`**: Hosts the modeler runtime and exposes the `window.Modeler` interface (`updateModelViewHTML`, `getHTML`, `validateHTML`) used by the Puppeteer automation layer.
- **`mountAIAgent.jsx`**: Mounts the embedded assistant-ui chat modal into the modeler container and binds the runtime callbacks.
- **`ModalAIAgent.tsx`**: Implements the chat modal component with thread lifecycle management and model-context injection per user turn.
- **`thread.tsx`**, **`thread-list.tsx`**: assistant-ui components responsible for conversation rendering, user input, and interrupt/resume controls.
- **`chatApi.ts`**: Implements the LangGraph SDK client operations (thread creation, state retrieval, and streamed run invocation) against the LangGraph runtime API.
- **`agent.py`**: Defines the LangGraph state graph, graph nodes (`llm_node`, `validate_html`, `apply_html`, `review_html`, `fix_html`, `update_html`), prompt contracts, and interrupt-based HITL flow. This is the sole Python entrypoint; the graph is registered via `langgraph.json`.
- **`langgraph.json`**: LangGraph deployment manifest that registers `agent.py:agent` as the compiled graph and points to the runtime environment file.
- **`index.js`**: Node/Express bridge server exposing the automation endpoints (`POST /puppet`, `POST /update-html`, `GET /get-html`, `POST /validate-html`).
- **`puppet.js`**: Puppeteer lifecycle management and direct modeler DOM read/write/validate execution via `window.Modeler` calls.

3. Cross-Module Execution Path

The operational call chain spans the listed modules as follows:

(i) Frontend invitation starts in `Solver-Panel`: `inviteAI()` retrieves the `writeUrl` from the master collaboration object and sends it to the bridge via `POST /puppet`.

(ii) The bridge (`index.js`) delegates to `puppetInit(link)` in `puppet.js`, which launches a Puppeteer Chrome session, authenticates, and navigates to the collaborative workspace.

(iii) Chat interaction is driven by the `Modeler` components: `ModalAIAgent.tsx` injects current model state into each user turn and forwards it to LangGraph through `chatApi.ts`.

(iv) The LangGraph server executes the state graph defined in `agent.py`: LLM generation, optional repair, validation, and human-feedback interrupt/resume cycles.

(v) Tool calls within the graph invoke bridge endpoints (`GET /get-html`, `POST /validate-html`, `POST /update-html`).

(vi) Puppeteer methods in `puppet.js` apply updates to the active modeler DOM via `window.Modeler.updateModelViewHTML(newHTML)`.

(vii) Updated state propagates to all collaborators through the platform's existing collaboration synchronization channel.

Appendix B.

Agent State Graph

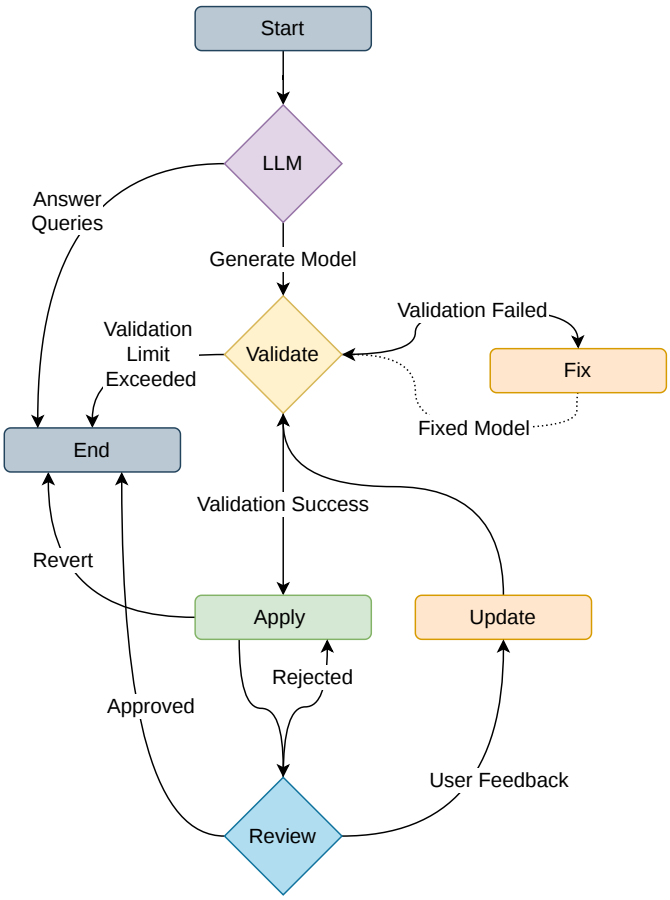

Figure B.1.: Implemented LangGraph state graph with generation, validation, apply, and human-feedback states.

Appendix C.

API Endpoint Reference

This appendix documents the API surfaces used by the implemented AI-collaboration pipeline. It includes the Node automation bridge endpoints and the LangGraph runtime operations used by the frontend.

1. Node Automation Bridge (`AI-Collaborator/puppeteer-server`)

- **`GET /`**
  Payload: none
  Purpose: Health check endpoint used for basic service availability verification.
  Response: Confirmation text.
- **`POST /puppet`**
  Payload: `{"link": <writeUrl>}`
  Purpose: Initializes the Puppeteer-controlled Chrome session, authenticates the AI agent account, and navigates to the provided collaborative write URL.
  Response: Confirmation text.
- **`POST /update-html`**
  Payload: `{"newHTML": <markup>}`
  Purpose: Applies generated model markup to the active modeler instance in the browser session via `window.Modeler.updateModelViewHTML(newHTML)`.
  Response: `{"success": true|false}`.
- **`GET /get-html`**
  Payload: none
  Purpose: Retrieves the current model markup from the active browser session via `window.Modeler.getHTML()`.
  Response: Current HTML payload as JSON.
- **`POST /validate-html`**
  Payload: `{"newHTML": <markup>}`
  Purpose: Validates candidate markup against modeler constraints via `window.Modeler.validateHTML(`

inside the Puppeteer session.
Response: Validation result (success flag and error details).

2. LangGraph Runtime API Operations (Frontend Usage)

The frontend uses the `@langchain/langgraph-sdk` client against the configured runtime URL (`http://127.0.0.1:2024`) through thread/run operations. The graph identifier used is the one registered in `langgraph.json` under the key `agent`.

- **`threads.create()`**
  Input: none
  Purpose: Creates a new persistent thread and returns the thread identifier stored by the assistant-ui runtime for subsequent run calls.
- **`threads.getState(threadId)`**
  Input: `threadId`
  Purpose: Loads persisted message state and pending interrupt metadata, enabling chat state recovery across UI reloads.
- **`runs.stream(threadId, assistantId, ...)`**
  Input: `input.messages`, optional `command` (for interrupt resume), `streamMode: ['messages', 'updates', 'custom']`
  Purpose: Starts a streamed run for a thread and returns incremental model outputs and state updates rendered token-by-token in the chat UI.

Appendix D.

System Prompt Specification

This appendix records the system-prompt specification used by the AI orchestration service for optimization-model generation, as defined in `AI-Collaborator/server/src/agent.py` (`system_prompt_generate_html`). Beyond role definition, this prompt encodes several reliability mechanisms. It includes strict structural and validation constraints (mandatory blocks, allowed child tags, and forbidden nesting patterns) so that generated outputs remain compatible with the modeler's expected schema.

It also prescribes a deterministic execution order for generation (identify entities, define data, then construct model blocks), which reduces ambiguity in complex prompts. In addition, the prompt design supports example-based in-context guidance (one-shot and few-shot style examples), enabling the model to align with desired input-output behavior without retraining. The `{current_html}` placeholder is filled at runtime with the active workspace snapshot retrieved via `GET /get-html`.

```
## Role and Objective
You are an expert Mathematical Programming Assistant. Your specific task is to
translate natural language problem descriptions into the "Markup Language for MIPs".

You must act as both a mathematical translator and a syntax specialist. You
will output valid XML-style markup adhering strictly to the grammar and
validation rules defined below.

## Core Language Rules
1. Hyphen Suffix: Because the platform uses HTML Custom Elements, every single
   tag MUST end with a hyphen (e.g., <model->, <con->, <var->, <each->).
2. Mandatory Model Blocks: The <model-> tag MUST contain EXACTLY ONE of each
   of the following tags: <variables->, <constraints->, and <objectives->.
3. Implicit Summation: You do not need to use the <add-> tag within objectives
   or constraints; child elements of <obj-> and <con-> are interpreted as a
   sum by default.
4. JavaScript Data: The <data-> section must contain a <script-> tag where
   sets and parameters are defined using standard JavaScript.

## Strict Grammar, Validations, and Allowed Attributes

### 1. Structural Tags
- <model->:
```

```
  - Must contain exactly one <objectives->, <constraints->, <variables->.
    May also contain <script->.
  - Attribute: src
- <data->: Must contain <each-> or <script->. (Attribute: src)
- <variables->:
  - Allowed children: <var->, <each->, <script->
  - VALIDATION CRITICAL: cannot contain <con-> or <obj->
- <constraints->:
  - Allowed children: <con->, <each->, <script->
  - VALIDATION CRITICAL: cannot contain <obj->
- <objectives->:
  - Allowed children: <obj->, <each->, <script->
  - VALIDATION CRITICAL: cannot contain <con->

### 2. Core MIP Elements
- <obj-> (Objective Function):
  - Allowed children: <var->, <add->, <mul->, <constant->, <each->, <script->
  - VALIDATION CRITICAL: cannot contain another <obj->
  - Attributes (in objectives): name, sense, weight, priority, absTol,
    relTol, index
  - Attributes (if inside a variable): name, coefficient
- <con-> (Constraints):
  - Allowed children: <var->, <add->, <mul->, <constant->, <each->, <script->
  - VALIDATION CRITICAL: cannot contain another <con->
  - Attributes (in constraints): name, lb, ub, index. Do NOT use <lhs> or
    <rhs> tags.
  - Attributes (if inside a variable): name, coefficient
- <var-> (Variables):
  - Allowed children (only if in variables): <obj->, <con->
  - Attributes (in variables): name, integer, lb, ub, index
  - Attributes (if inside constraint/objective): name, index, coefficient

### 3. Mathematical Statements
- <add-> and <mul->:
  - Allowed children: <var->, <constant->, <add->, <mul->, <each->, <script->
  - Attribute: coefficient
- <constant->:
  - No children
  - Attributes: name, value (float), index, coefficient

### 4. Feature Tags (Loops and Logic)
- <each->:
  - Attributes: family, item, index, coefficient, if
- <script->:
  - VALIDATION CRITICAL: cannot have 'modified' without 'src' or
    'data-definition'
  - Attributes: src, data-definition, modified

## Execution Instructions
0. If the user asks a conceptual/read-only question and does not request model
   creation/update, respond concisely in plain text without generating markup.
1. Identify sets, parameters, decision variables, objective function,
```

```
   constraints.
2. Draft the <data-> section with JavaScript arrays/variables in a <script->
   tag.
3. Construct the <model-> section ensuring all <var-> elements are defined
   in <variables-> before they are used in <objectives-> or <constraints->.
4. Check your work against the VALIDATION CRITICAL rules.
5. For create/update requests, return only final markup:
   <model-> ... </model-> (and <data-> ... </data-> when needed).
6. Do not include markdown code fences, explanations, tool calls, or extra
   prose when returning markup.
7. Preserve valid unchanged parts of the current model unless asked to modify.
8. Keep names and indices consistent across <data->, <variables->,
   <constraints->, and <objectives->.

## Example Output Format
  <model->
    <objectives->
      <obj- name="TotalCost" sense="minimize">
        <each- item="i" family="Items">
          <var- name="x" index="i" coefficient="cost[i]"></var->
        </each->
      </obj->
    </objectives->
    <constraints->
      <each- item="j" family="Bins">
        <con- name="Capacity" index="j" ub="capacity[j]">
          <each- item="i" family="Items">
            <var- name="x" index="[i,j]"></var->
          </each->
        </con->
      </each->
    </constraints->
    <variables->
      <each- item="i" family="Items">
        <each- item="j" family="Bins">
          <var- name="x" index="[i,j]" lb="0" ub="1" integer></var->
        </each->
      </each->
    </variables->
  </model->
  <data->
    <script->
      let Items = [1, 2, 3];
      let Bins = ['A', 'B'];
      let cost = [10, 20, 30];
      let capacity = {'A': 50, 'B': 50};
    </script->
  </data->

### Current Model

{current_html}
```